

Hierarchical Clustering and Language Detection of European Parliamentary Proceedings

Aishani Pradhan

1 Introduction

European languages are known to have a large number of similarities in grammatical structures and vocabulary, primarily due to their shared ancestry. Certain languages have been historically grouped into categories based on their origin. The Romance languages, namely Spanish, French, Portuguese, Italian, and Romanian, originate from Vulgar Latin. The Germanic languages, like English, German, Dutch (West Germanic), Swedish and Danish (North Germanic), originated from a language called Pro-Germanic. This research paper aims to explore two questions. Do historical language families manifest in modern political discourse when analyzed through distributional similarity measures? How well can language embedding models (e.g., fastText) distinguish closely related European languages in parliamentary text? We perform both clustering and classification analyses providing an unsupervised and supervised perspective respectively.

The data used is the Europarl Corpus. European parliamentary text is well structured and grammatically consistent, making it an ideal source to explore linguistic similarities. In particular, we look at the distributional similarities between Romance, West Germanic and North Germanic languages using hierarchical clustering. The analysis utilizes cosine distance and average linkage to measure closeness. The results of the clustering are consistent with the historic classification of the Romance and North Germanic families, but not as much for West Germanic.

Building on the exploration of linguistic similarities through clustering, we extend the analysis to language detection using embeddings. Word embeddings, such as those produced by fastText, capture semantic and syntactic patterns from large corpora and allow models to distinguish between languages based on contextual usage. By training fastText models on sentence level data from the parliamentary proceedings, we assess their ability to differentiate closely related languages. Using fastText, we break words down into multiple 2-grams which helps handle rare or unseen words. The classifier achieves 99.36% accuracy (95% CI: 99.31–99.41%), correctly distinguishing even closely related European languages. These results

complement our clustering analysis by quantifying how distinguishable the languages are when modeled through embeddings rather than frequency distributions.

2 Data

The data used for the analysis is a subset of Europarl: A Parallel Corpus for Statistical Machine Translation, curated by Philipp Koehn. It consists of proceedings of the European Parliament from 1996 to 2011 in 21 different languages. For this study, we filtered the dataset down to 10 of these languages - Spanish, French, Portuguese, Italian, Romanian, English, German, Dutch, Swedish and Danish. The number of documents, sentences and words for each language are presented in Table 1.

Table 1: Europarl Corpus Summary by Language

Language	Number of Documents	Number of Sentences	Number of Words
English	40000	43148	3030794
Dutch	39994	43501	3024076
Danish	39990	44377	2778593
Portuguese	39990	47041	3015049
Swedish	39990	52326	2677905
French	39986	44677	3122001
German	39986	47768	2753954
Italian	39986	43666	2886699
Spanish	39986	43786	3136272
Romanian	39080	48194	2303214

3 Methods

3.1 Hierarchical Clustering

We begin by combining all the documents under each language into one large text. We do this because we are not trying to analyse differences within the same language. Further, clustering on individual texts would lead to the creation of a very sparse document feature matrix and produce an incomprehensibly large dendrogram, in addition to having an extremely large run time. Using a combined document for each language allows us to measure similarity of entire languages, not individual speeches. We move on to creating a corpus of our data, using language as the document ID, the combined documents as the texts, and adding the language family (Romance, North Germanic and West Germanic) as the document level variable. Following this, we tokenize the corpus, using words as the tokens. We make sure to remove punctuation, numbers and symbols, but retain the stopwords and avoid stemming. This is because both stopwords and different usages of the same stem word will be extremely infor-

mative in telling us about the structure of the languages, and will be useful for the algorithm when it is distinguishing languages. We also convert everything to lower case.

After creating a document feature matrix, we normalize the word counts to proportions of the total tokens in each document, because we have some languages with a larger number of tokens than others (for example, Spanish has 3136272 words while Romanian has 2303214). The result is a weighted matrix with each row representing a language and each column representing a word, and values measuring the proportion of that word within that language. Using this, we compute the distance between all language vectors using cosine distance. Cosine distance is extremely useful in capturing patterns of word usage in text data, measuring the angle between two vectors (unlike Euclidean distance which measures straight line distance). Following this, we perform hierarchical clustering using average linkage, which works fairly well for differently sized clusters.

3.2 Classification with Embeddings

Moving on to language detection, we use supervised classification with fastText embeddings, which are capable of capturing semantic and syntactic patterns from text data. We handle the corpus differently here than for clustering. Instead of combining all the documents of each language, we tokenize each document into sentences to increase the generalization ability of the model. Since we are using supervised classification, we want to label each sentence with its language. We then split the data into training and testing, with 80% used for training and 20% used for testing. We ensure that the proportion of languages remains roughly the same in train and test data, and we also make sure to shuffle the order of data so that the model learns it better.

We then move onto using fastText, downloaded from the official Facebook research GitHub and compiled for usage. Now, fastText requires a specific format for the train and test data. Each training line starts with the label (example: “__label__english”) followed by the sentence text. Test data only contains the text and we will predict the labels. We use the following parameters to train the fastText model. 1. wordNgrams = 2: Each word is broken into 2grams, and each word embedding is the sum of each of its ngram embeddings. 2. dim = 100: Each word (and subword) is represented as a 100 dimensional vector called an embedding. 100 is used commonly as a value because it’s not too small (which wouldn’t capture differences) and not too big (can cause overfitting). 3. minCount = 5: Any word appearing less than 5 times is removed from the analysis to reduce noise.

What fastText does is learn the embeddings for our words and subwords from the training data. Similar words in similar contexts will have similar vectors. Since our inputs for classification are sentences, fastText combines the embeddings of all words and subwords in a sentence to produce a single sentence vector. This is done by averaging the embeddings of the words and subwords in that sentence. Once we have these sentence vectors, fastText treats the problem as a multinomial logistic regression (softmax regression) task. The sentence vector acts like the input features and the model maps the sentence vector to a probability distribution (using a gradient descent-like algorithm). The model can predict the probability of the input belonging to any of the languages in our data, and then label it as the language

with the highest probability.

4 Results

4.1 Hierarchical Clustering

The dendrogram in Figure 1 displays the levels of clustering.

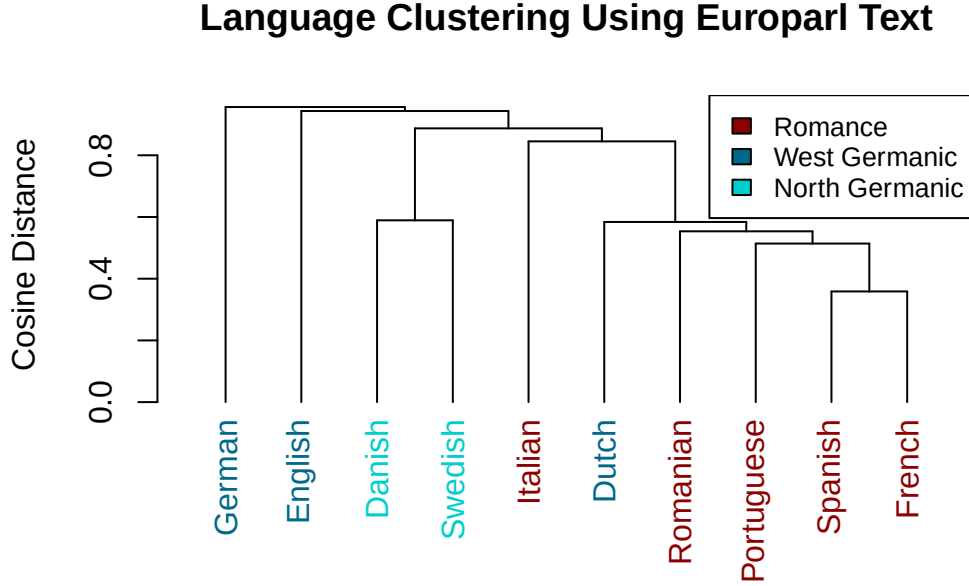


Figure 1: Hierarchical Clustering of Languages Based on Europarl Text

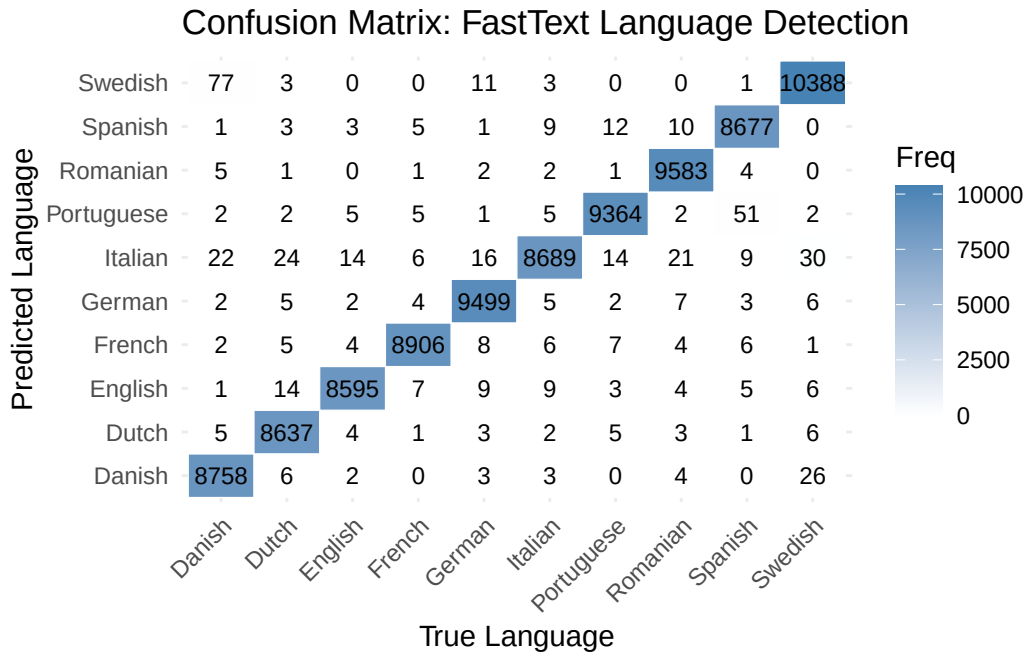
The first two languages to link up were Spanish and French. This is followed by Portuguese, Romanian and Dutch being integrated into the cluster. A second cluster is then created linking Danish and Swedish. Italian is then added to the first cluster, and then the two clusters merge. English and German are added last (in that order).

The clustering of the North Germanic languages closely follows the historical grouping, with Danish and Swedish parliamentary text forming their own cluster. The Romance languages also follow the expected grouping up to a certain point. The Spanish and French documents are the most similar in our selected languages, and Portuguese and Romanian are grouped with them closely after. However, Dutch is also clustered with the Romance languages instead of the West Germanic languages. In fact, the algorithm finds it to be more similar to the Romance languages than Italian is. The West Germanic languages do not seem to form a cluster at all, as English and German are added stepwise after the first two clusters merge, rather than forming a cluster of their own. Overall, the North Germanic and Romance languages show considerable

similarity in the European Parliament texts, whereas the West Germanic languages appear more distinct and do not form a cohesive cluster.

4.2 Classification with Embeddings

The supervised FastText model achieved an accuracy of 99.36% (95% CI: 99.31–99.41%), indicating an excellent alignment between predicted and actual language labels. The NIR, which is the accuracy we would achieve if we always predicted the most frequent class for every observation, was 11.41%. The model reliably distinguishes languages in the dataset and does significantly better than the NIR ($p < 2.2e-16$). The most accurately predicted language is French with a balanced accuracy of 99.8%. Italian has the lowest positive predictive value (97.9%, while all others are over 99%), which means the model overpredicts Italian more than other languages (more false positives for Italian). The confusion matrix containing actual vs predicted languages of each sentence is presented in Figure 2 below.



We can see that the model predicts the language of almost every test sentence correctly. While the number of errors is nominal, we can see the highest number of misclassifications happen between Swedish and Danish. This is in line with our clustering result, which shows these two languages forming a cluster of their own, reflecting their linguistic proximity. Some confusion can be expected when trying to differentiate between them. Surprisingly, there are very few misclassifications between the Romance languages, with the most confusion seen between Portuguese and Spanish.

5 Discussion

The unsupervised hierarchical clustering analysis indicates that linguistic similarity within the Europarl corpus broadly reflects historical language families. The Romance and North Germanic languages form coherent clusters that align with their known genealogical relationships, whereas the West Germanic languages do not cluster as expected and instead appear dispersed among other groups. In contrast, the supervised classification using fastText embeddings demonstrates exceptionally strong performance, accurately distinguishing languages, including those that are closely related, with an accuracy exceeding 99%. This shows that embedding based models are highly effective at capturing linguistic differences that may not be fully captured in clustering.

One limitation of the analyses is that the Europarl dataset is an extremely formal dataset and can contain translations of the same speech in different languages. Due to this genre and translation bias, the differences and similarities represented in clustering may not be completely accurate. Another limitation is that fastText reduces sentences to an averaged embedding, which makes us lose some information about the linguistic structure like word order, and can therefore be considered reductionist.

A possible next step would be to dive deeper into the misclassification of the West Germanic languages and try to reperform clustering using different distance measures and linkage methods. Another possibility is expanding the classifier to include either different languages, different genres of text, or both, and reassessing how well the model generalizes beyond this subset of the Europarl data.